

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #10 - Semana 11 – Sivarama - Capítulo 17
Procesamiento de cadenas

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 07/10/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 363–378.

Abstract—String manipulation is a fundamental task in programming, and in the IA-32 architecture, it is supported by a specialized set of instructions. Strings can be represented with fixed or variable length, each with its own advantages and limitations. The IA-32 provides five main string instructions (LODS, STOS, MOVS, CMPS, SCAS) that allow copying, comparing, and searching within strings. These instructions are versatile, allowing the manipulation of other types of data, such as arrays. They can also be combined with repetition prefixes to enhance their functionality. Additionally, indirect procedure calls provide flexibility in dynamic function handling.

I. Procesamiento de cadenas (strings)

En la arquitectura IA-32, existe un conjunto de instrucciones especializadas para manipular cadenas. Estas instrucciones permiten realizar operaciones como copiar, comparar o buscar en cadenas. A pesar de que están orientadas a cadenas, también se pueden utilizar para procesar otros tipos de datos, como arreglos de palabras dobles. Este capítulo ilustra cómo usar estas instrucciones y presenta un programa de ejemplo que las prueba, incluyendo el uso de llamadas indirectas a procedimientos.

II. Representación de cadenas

Las cadenas pueden representarse de dos maneras: con longitud fija o longitud variable.

A. Cadenas de longitud fija

Cada cadena ocupa la misma cantidad de caracteres, incluso si algunas cadenas son más cortas. Para ajustarse al espacio disponible, las cadenas se rellenan con

caracteres adicionales o se truncan. Esto puede llevar a un desperdicio de memoria si la mayoría de las cadenas son más cortas que el espacio asignado.

B. Cadenas de longitud variable

En este esquema, las cadenas pueden tener una longitud dinámica, asociada con un atributo de longitud. La longitud puede almacenarse explícitamente o mediante un carácter sentinela, como el carácter NULL (00H) al final de la cadena, como en las cadenas terminadas en NULL de C.

III. Instrucciones de procesamiento de cadenas

El conjunto de instrucciones IA-32 incluye cinco principales instrucciones para manipular cadenas: **LODS**, **STOS**, **MOVS**, **CMPS** y **SCAS**. Estas instrucciones no se limitan al procesamiento de cadenas, también son útiles para copiar arreglos de datos. Las operaciones se llevan a cabo utilizando los registros **ESI** y **EDI** para apuntar a las posiciones de origen y destino en memoria. Cada instrucción puede operar en datos de 8, 16 o 32 bits, y se pueden aplicar con prefijos de repetición para repetirlas de acuerdo con el contenido del registro **ECX**.

A. LODS (Cargar cadena)

Esta instrucción carga el valor de la cadena fuente en un registro (AL, AX o EAX), actualizando el registro **ESI** según la dirección de procesamiento.

Ejemplo de ensamblador:

```
lodsb ; carga un byte de la cadena fuente en AL
```

B. STOS (Almacenar cadena)

STOS almacena el valor de AL (o AX, EAX) en la cadena destino, actualizando **EDI** según la dirección de procesamiento.

Ejemplo de ensamblador:

```
stosb ; almacena un byte en la cadena destino
```

C. MOVS (Mover cadena)

MOVS copia un valor (byte, palabra o doble palabra) de la cadena fuente a la cadena destino, actualizando los registros **ESI** y **EDI**.

Ejemplo de ensamblador:

```
movsb ; copia un byte de la cadena fuente a la cadena destino
```

D. CMPS (Comparar cadenas)

CMPS compara los valores de las dos cadenas en las posiciones de origen y destino, y ajusta los indicadores de acuerdo con el resultado.

Ejemplo de ensamblador:

```
cmpsb ; compara dos cadenas byte por byte
```

E. SCAS (Buscar en cadena)

SCAS busca un valor específico en la cadena destino, comparando el valor en AL (o AX, EAX) con el contenido de la cadena en **EDI**.

Ejemplo de ensamblador:

```
scasb ; busca un byte en la cadena destino
```

IV. Prefijos de repetición

Las instrucciones de cadena pueden repetirse utilizando prefijos de repetición como **REP**, **REPE/REPZ** y **REPNE/REPNZ**, que repiten la operación en función del contenido del registro **ECX** o de las condiciones del indicador de cero (ZF). Por ejemplo, **REPNE CMPSB** compara los bytes de dos cadenas hasta que encuentra una diferencia o se agota **ECX**.

V. Banderas de dirección

El procesamiento de cadenas puede realizarse hacia adelante o hacia atrás, dependiendo del estado del indicador de dirección (**DF**). Si **DF** está despejado (0), las operaciones avanzan en la cadena; si está establecido (1), las operaciones retroceden. Las instrucciones **STD** y **CLD** permiten controlar la dirección de procesamiento.

Ejemplo de ensamblador para establecer la bandera de dirección:

```
std ; establece la dirección hacia atrás
cld ; establece la dirección hacia adelante
```

VI. Llamadas indirectas a procedimientos

En las llamadas a procedimientos indirectos, la dirección del procedimiento no se especifica directamente, sino que se almacena en un registro o en una ubicación de memoria. Esto permite una mayor flexibilidad en la gestión de procedimientos.

Ejemplo de ensamblador:

```
call EBX ; llama al procedimiento cuya dirección está en EBX
call [proc_ptr] ; llama al procedimiento apuntado por proc_ptr
```